

University of Engineering and Technology, Lahore



MR_Lab_Report_8

Submitted by: 2022_MC_27

Lab Session 8: Building and Visualizing a Custom Mobile Robot using URDF

Objective

- Understand the **Unified Robot Description Format (URDF)** and its role in ROS 2 robot modeling.
- Design a **custom mobile robot model** using links and joints in URDF.
- Build a **robot structure** including base chassis, wheels, and sensor links.
- Learn how to define and organize robot components such as **links, joints, and materials**.
- Visualize the robot model in **RViz using ROS 2 tools**.
- Inspect and understand coordinate relationships using the **TF Tree via tf2_tools**.
- Gain hands-on experience in preparing a robot description for **simulation in Gazebo**.
- Develop a foundational understanding for future work in **robot simulation and control systems**.

Introduction

In modern robotics, before a robot is physically built or simulated, it must be described mathematically and geometrically in a format that software tools can interpret. The Unified Robot Description Format (URDF) is the standard way to do this within the ROS (Robot Operating System) ecosystem. URDF uses XML to define a robot as a tree of rigid bodies called links, connected by joints that specify how those bodies relate to and move relative to one another.

Each link can contain visual geometry (what the robot looks like), collision geometry (used for physics simulation), and inertial properties (mass and moment of inertia for dynamics). Joints define the type of motion allowed between two links — for example, a fixed joint allows no movement, a revolute joint allows rotation within limits, and a continuous joint allows unlimited rotation, which is ideal for drive wheels.

ROS 2 provides a set of tools that work directly with URDF files. The `robot_state_publisher` node reads the URDF and continuously broadcasts coordinate frame transforms (TF) for every link, allowing the rest of the ROS 2 system to always know where each part of the robot is in space. RViz, the ROS visualization tool, consumes these transforms and renders the robot in an interactive 3D environment. The `tf2_tools` package can then generate a TF Tree diagram, giving a clear picture of the parent-child relationships between all robot frames.

This lab builds practical skills in XML-based robot modeling, ROS 2 workspace management, and 3D visualization — all of which are essential competencies for robotics engineers working on real hardware or simulation-based development.

Lab Summary

This lab session focused on learning how to describe and visualize a mobile robot using URDF (Unified Robot Description Format) within the ROS 2 ecosystem. URDF is an XML-based format that allows engineers and students to define the physical structure of a robot — its links (rigid bodies) and joints (connections between links) — in a way that ROS 2 tools can understand and render.

The session began by setting up the necessary ROS 2 packages, including `urdf_tutorial` for visualization and `tf2_tools` for inspecting coordinate frame relationships. Students created a structured workspace with dedicated folders for the URDF file, launch configurations, and RViz settings. The robot was then launched in RViz using the `display.launch.py` script, where the robot model appeared in a 3D viewer and joint sliders allowed real-time interaction.

A key conceptual tool explored in this lab was the TF Tree, generated by running `view_frames` from the `tf2_tools` package. This tree visualizes how every link in the robot is connected through transforms, confirming that the robot's structure is logically consistent and that ROS 2 is correctly tracking every frame.

Summary of Custom Robot Design

Building upon the basic template provided in the lab guide, a fully custom differential-drive mobile robot was designed and described in URDF. The robot incorporates multiple geometric primitives, precise joint placement, and a clean visual style using a yellow-and-black color scheme.

The custom robot includes the following components:

- Rounded-corner base chassis — achieved by combining two overlapping box shapes (0.34 x 0.30 x 0.10 m and 0.40 x 0.24 x 0.10 m) with four corner cylinders (radius 0.03 m) placed at the front-left, front-right, back-left, and back-right corners. This gives the illusion of a body with rounded edges, all colored yellow.
- Camera link — a yellow cylinder (radius 0.05 m, length 0.05 m) mounted centrally on top of the base via a fixed joint. It is positioned 0.075 m above the base center, representing a sensor or camera turret.
- Left and right drive wheels — two dark-black cylinders (radius 0.05 m, length 0.02 m) attached using continuous joints, allowing free rotation. They are placed symmetrically on either side of the rear portion of the chassis ($y = \pm 0.15$ m) and rotated 90° to lie flat as wheels.
- Front caster wheel — a dark-black sphere (radius 0.025 m) fixed to the front underside of the base ($x = 0.15$ m, $z = -0.075$ m), acting as a passive support wheel for balance.

URDF Code Explanation

The URDF file is structured as a standard XML document wrapped in a `<robot>` tag. Each physical part of the robot is declared as a `<link>`, and the relationships between parts are declared as `<joint>` elements.

Materials

Two named materials are defined at the top of the file:

```
<material name="yellow"><color rgba="1.0 0.85 0.0 1.0"/></material>
<material name="dark_black"><color rgba="0.02 0.02 0.02 1.0"/></material>
```

These are referenced by name in every visual element, keeping the color definitions centralized. The rgba values represent Red, Green, Blue, and Alpha (opacity) channels, each ranging from 0 to 1.

Base Link (Rounded Chassis)

The `base_link` is the root of the robot's TF tree — it is the reference frame from which all other links are measured. To create the appearance of a rounded body, six visual elements are combined inside a single link: two overlapping boxes form the cross-shaped body, and four cylinders are placed at the corners. No `<collision>` or `<inertial>` tags are included because this lab focuses only on visualization.

Joints

Three types of joints are used in this robot:

- `fixed` — the `camera_joint` and `caster_wheel_joint` are fixed, meaning they cannot move. The child link is rigidly attached to the parent at a specified offset and rotation (origin xyz and rpy).
- `continuous` — the `left_wheel_joint` and `right_wheel_joint` are continuous, meaning they can rotate indefinitely around an axis. In RViz, the joint state publisher slider allows these to be animated. This is the joint type used for motorized wheels in a differential-drive robot.

Each joint specifies a `<parent link>` and a `<child link>`, along with an `<origin>` that defines where the child is placed relative to the parent. The `rpy="1.57 0 0"` on the wheel joints rotates them 90° around the X-axis so the cylinders stand upright as wheels rather than lying flat as discs.

Camera Joint

```
<joint name="camera_joint" type="fixed">
<parent link="base_link"/>
<child link="camera"/>
<origin xyz="0 0 0.075" rpy="0 0 0"/>
</joint>
```

This places the camera 0.075 m above the center of the base, sitting flush on top of the chassis.

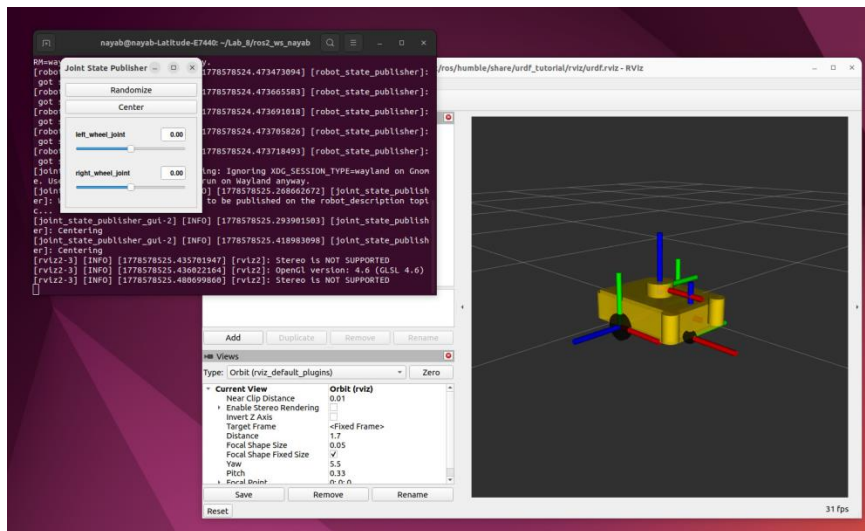
Caster Wheel Joint

```
<joint name="caster_wheel_joint" type="fixed">
<parent link="base_link"/>
<child link="caster_wheel"/>
<origin xyz="0.15 0 -0.075" rpy="0 0 0"/>
</joint>
```

The caster is placed 0.15 m forward and 0.075 m below the base center, so it touches the ground when the robot is standing on its wheels.

Customizations Made

- Replaced the simple flat box base with a rounded-corner approximation using two boxes and four corner cylinders for a more realistic robot body shape.
- Used a yellow and dark-black color scheme throughout, giving the robot a distinctive visual identity in RViz.
- Added a front caster wheel (sphere geometry) to complete the three-point ground contact needed for a stable differential-drive robot.
- Positioned the drive wheels at the rear section of the chassis ($x = -0.05$ m) and the caster at the front ($x = 0.15$ m), matching a realistic differential-drive layout.
- Set the camera link as a cylinder on top of the chassis to represent a sensor payload, connected via a fixed joint with a precise vertical offset.



Conclusion

- Successfully designed and built a custom differential-drive mobile robot using URDF, combining boxes, cylinders, and spheres to form a realistic rounded-chassis robot with a yellow-and-black color scheme.
- Learned how URDF uses a parent-child link-joint tree structure to define every part of a robot, and how each joint origin must be precisely calculated relative to its parent link for the robot to assemble correctly in 3D space.
- Visualized the robot in RViz and confirmed that all links, joints, and geometries rendered correctly with the robot_state_publisher broadcasting transforms for every frame.
- Used tf2_tools to generate the TF Tree, which validated the full parent-child frame hierarchy from base_link through the wheels, caster, and camera.
- Understood the difference between joint types: fixed joints (camera, caster) allow no movement, while continuous joints (left and right wheels) allow unlimited rotation and can be animated using RViz joint sliders.
- Gained practical experience in ROS 2 workspace organization, including creating dedicated urdf/, launch/, and rviz/ folders and building the package with colcon build.

- This lab lays the complete foundation for the next session, where the same URDF model will be loaded into Gazebo for physics-based simulation, with plans to add LiDAR sensors, collision geometry, and inertial properties.